Compito 2.1

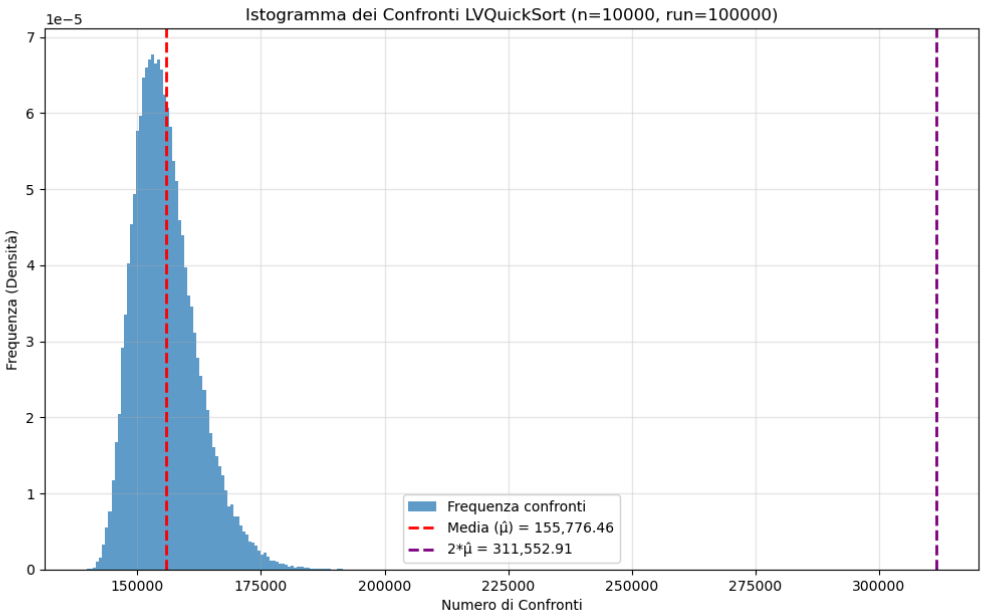
Malchiodi Riccardo 5680500

Obiettivo

Il nostro obiettivo in questo compito è quello di andare a osservare la distribuzione del numero di confronti effettuati da **LVQuicksort** su una sequenza **S** di numeri con $|S| = 10^4$. Attraverso l'esecuzione di 10^5 run indipendenti, calcoleremo la media empirica **μ^** e produrremo un istogramma delle frequenze per visualizzare la concentrazione dei valori attorno alla media. Successivamente, verificheremo la precisione delle disuguaglianze probabilistiche (confrontando la stima teorica con quella empirica) e utilizzeremo i risultati ottenuti per calcolare la probabilità di successo della variante **MCQuickSort**.

Analisi Output

LasVegas QuickSort



L'istogramma seguente riporta la distribuzione di frequenza del numero di confronti effettuati dall'algoritmo **LVQuickSort** su 10^5 esecuzioni indipendenti (*run*), applicate a una sequenza **S** di dimensione $|S| = 10^4$. Sull'asse delle ascisse è rappresentato il numero di confronti, mentre l'asse delle ordinate indica la densità di frequenza. Sono state inoltre evidenziate due soglie critiche:

- La media empirica μ^ ≈ 155.776 (linea tratteggiata rossa).
- Il doppio della media empirica 2μ^ ≈ 311.536 (linea tratteggiata viola).

Emerge chiaramente che la distribuzione dei confronti è fortemente concentrata attorno alla media μ^. La curva assume una forma “a campana” molto stretta. È significativo notare che la totalità delle osservazioni ricade ben al di sotto della soglia 2μ^. Infatti, la regione del grafico in corrispondenza della linea viola è completamente vuota, indicando che in nessuno dei 10^5 run l’algoritmo si è avvicinato a tale numero di confronti.

Verifica della Disuguaglianza(3)

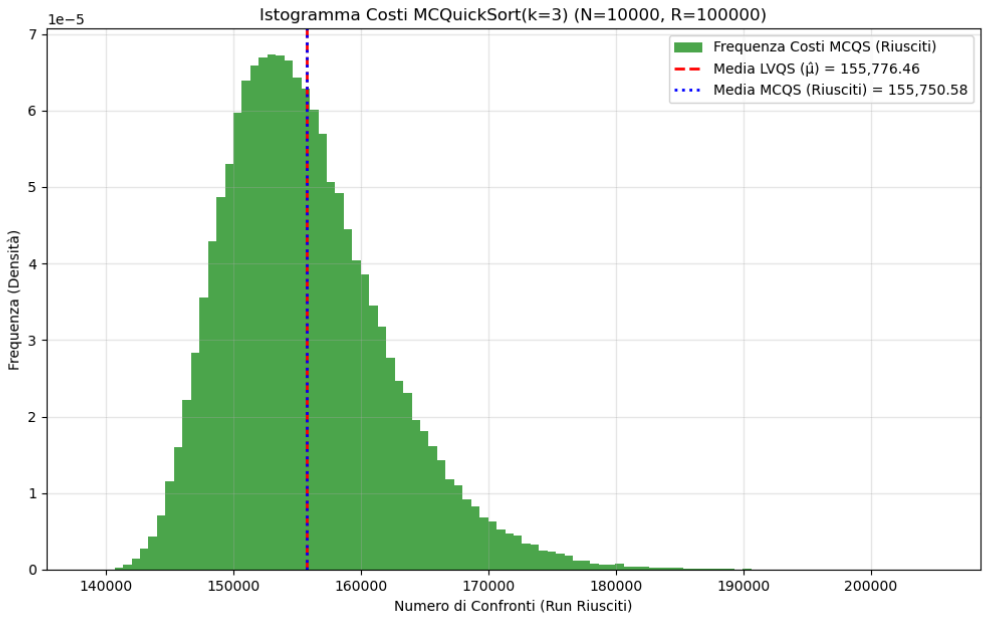
L’obiettivo dell’esperimento era verificare se la disuguaglianza di Markov, che fornisce un limite superiore di probabilità $P(X \geq 2\mu\hat{}) \leq 1/2$, fosse eccessivamente pessimista. Per fare ciò, abbiamo testato la disuguaglianza (3):

$$(P(X \geq 2\mu\hat{}) \leq \epsilon)$$

ponendo $\epsilon = 10^{-5}$.
Analizzando i dati sperimentali:

- il numero di run in cui i confronti hanno superato $2\mu\hat{}$ è risultato essere 0
- la probabilità empirica è dunque 0/10^5 = 0
- essendo 0 < 10^{-5} la disuguaglianza (3) è verificata.

MonteCarlo QuickSort



Questo istogramma mostra la distribuzione di frequenza del numero totale di confronti per l'algoritmo MCQuickSort con k = 3. Anche in questo caso abbiamo 10^5 esecuzioni indipendenti (*run*), applicate a una sequenza S di dimensione $|S| = 10^4$.

- **Barre verdi:** rappresentano la frequenza dei costi nei casi in cui l'algoritmo ha completato l'ordinamento entro il budget.

- **Linee Verticali:** Sono riportate la media originale di LVQuickSort (rossa tratteggiata) e la nuova media calcolata sui run riusciti di MCQuickSort (blu punteggiata).

Calcolo della Probabilità di Successo per MCQuickSort(S, 3)

L'esperimento condotto su **MCQuickSort** ha avuto come obiettivo quello di trasformare l'algoritmo **Las Vegas** (che garantisce la correttezza ma non il tempo di esecuzione) in un algoritmo **Monte Carlo**(garantisce il tempo di esecuzione con una piccola probabilità di fallire). Il nostro obiettivo ora è quello di calcolare la probabilità che l'algoritmo **MCQuickSort** ordini correttamente la sequenza **S** eseguendo massimo **k = 3** tentativi basandosi sulla **disugaglianza(3).**

Usando la disuguaglianza(3) sappiamo che: $p = P(X \geq 2\mu\text{^}) \leq \epsilon = 10^{-5}$.

<u>NOTA:</u> p corrisponde alla probabilità di un singolo tentativo di fallire

L'algoritmo **MCQuickSort** esegue fino a 3 tentativi indipendenti, l'algoritmo fallisce complessivamente se e solo se tutti e i 3 tentativi falliscono. Poiché i run sono indipendenti, la probabilità che falliscono tutti e 3 consecutivamente è il prodotto delle probabilità dei singoli fallimenti. Quindi possiamo definire: $P(Fallimento\ Totale) \leq (10^{-5})^3 = 10^{-15}$.
La probabilità di successo è definita da: $P(Successo) = 1 - P(Fallimento\ Totale)$ e quindi : $P(Successo) \geq 1 - 10^{-15}$

Codice

```
import random
import numpy as np
import matplotlib.pyplot as plt

# Parametri
DIMENSION = 10**4        # Dimensione della sequenza S
NUM_RUNS = 10**5         # Run per la media empirica LV
MC_RUNS = 100_000        # Run per l'esperimento MC
K_TENTATIVI = 3

def partition(arr, low, high, counter):
   pivot = arr[low]
   i = low
```

```python
    # Conteggio confronti: (high - low) elementi confrontati con il pivot
    num_comparisons = high - low
    if num_comparisons > 0:
        counter[0] += num_comparisons

    for j in range(low + 1, high + 1):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[low], arr[i] = arr[i], arr[low]
    return i

def lv_quicksort(arr, low, high, counter):
    if low >= high:
        return
    # Random Pivot (Las Vegas)
    pivot_index = random.randint(low, high)
    arr[low], arr[pivot_index] = arr[pivot_index], arr[low]

    split_index = partition(arr, low, high, counter)

    lv_quicksort(arr, low, split_index - 1, counter)
    lv_quicksort(arr, split_index + 1, high, counter)

# Funzione MCQuickSort
def mc_quicksort_experiment(S_original, k, mu_hat_budget, lv_func):
    n = len(S_original)
    for _ in range(k):
        S_copy = S_original.copy()
        attempt_counter = [0]
        # Esegui LVQuickSort
        lv_func(S_copy, 0, n - 1, attempt_counter)
        count = attempt_counter[0]

        # Se rientra nel budget, ritorna successo
        if count <= mu_hat_budget:
            return (True, count)
    # Se il loop finisce, tutti i tentativi sono falliti
    return (False, 0)


# PARTE 1: LVQuickSort (Generazione Dati + Istogramma)

comparison_counts = []
```

```python
# Esecuzione dei Run LV
for _ in range(NUM_RUNS):
    # Genera permutazione casuale
    S = list(np.random.permutation(DIMENSION))
    current_run_counter = [0]
    lv_quicksort(S, 0, len(S) - 1, current_run_counter)
    comparison_counts.append(current_run_counter[0])

# Calcolo statistiche per il grafico
mu_hat = np.mean(comparison_counts)
due_mu_hat = 2 * mu_hat

# Generazione Istogramma 1
plt.figure(figsize=(12, 7))
plt.hist(comparison_counts, bins=100, density=True, label='Frequenza confronti',
alpha=0.7)
plt.axvline(mu_hat, color='red', linestyle='dashed', linewidth=2, label=f'Media
({mu_hat:,.2f})')
plt.axvline(due_mu_hat, color='purple', linestyle='dashed', linewidth=2,
label=f'2*Media ({due_mu_hat:,.2f})')
plt.title(f'Istogramma dei Confronti LVQuickSort (n={DIMENSION}, run={NUM_RUNS})')
plt.xlabel('Numero di Confronti')
plt.ylabel('Frequenza (Densità)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# PARTE 2: MCQuickSort (Generazione Dati + Istogramma)

S_per_MC = list(range(DIMENSION)) # Sequenza base per i test MC
mc_success_costs = []
MU_HAT_BUDGET = 2 * mu_hat

# Esecuzione dei Run MC
for _ in range(MC_RUNS):
    success, cost = mc_quicksort_experiment(S_per_MC, K_TENTATIVI, MU_HAT_BUDGET,
lv_quicksort)
    if success:
        mc_success_costs.append(cost)

# Generazione Istogramma 2
plt.figure(figsize=(12, 7))
plt.hist(mc_success_costs, bins=100, density=True, label='Frequenza Costi MCQS
(Riusciti)', alpha=0.7, color='green')
```

```
plt.axvline(mu_hat, color='red', linestyle='dashed', linewidth=2, label=f'Media
LVQS ({mu_hat:,.2f})')

if mc_success_costs:
   mu_mcqs = np.mean(mc_success_costs)
   plt.axvline(mu_mcqs, color='blue', linestyle='dotted', linewidth=2,
label=f'Media MCQS Riusciti ({mu_mcqs:,.2f})')

plt.title(f'Istogramma Costi MCQuickSort (k={K_TENTATIVI}) (N={DIMENSION},
R={MC_RUNS})')
plt.xlabel('Numero di Confronti (Run Riusciti)')
plt.ylabel('Frequenza (Densità)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```